

Manual de instalación y uso

Autoras: Antonella Cuvertino y Miriam Hammami.

Qué vas a abrir

CAT-IA es una web local para un operador de soporte. La ventana de terminal mantiene el servidor activo; el navegador muestra la aplicación. Cerrar el navegador no detiene el servidor. Ctrl+C en la terminal lo detiene.

Instalación desde cero

Instala Python 3.12, habilita Python en PATH y extrae el ZIP en una carpeta de trabajo. Ejecuta `instalar.bat`. Esto crea `.venv`, instala `requirements.txt` y copia `.env.example` solo si no existe `.env`. Luego ejecuta `iniciar.bat` y abre `http://127.0.0.1:8000`. No compartas `.venv` entre equipos: créala en cada uno. No se necesitan Node, Docker ni bases de datos para la ruta por defecto.

Una instalación nueva desde el ZIP comienza en DEMOSTRACIÓN. Es útil para aprender la interfaz y funciona con las fuentes incluidas sin inferencia. No presentes esta etiqueta como modelo real. En la carpeta original de este equipo se dejó `.env` configurado para Ollama local; consulta su etiqueta antes de presentar.

LLM local para la evaluación

En este equipo se preparó Ollama portable dentro de `.tools`. Para reiniciar tras apagar el computador, ejecutar `iniciar_modelo_local.bat` y luego `iniciar.bat`. La configuración local utiliza el puerto 11435 y el modelo `qwen2.5:1.5b`. Esa carpeta pesada se excluye del ZIP. En un equipo nuevo se aplica la instalación estándar siguiente y el puerto 11434.

Descarga Ollama desde su sitio oficial, instala y abre la aplicación. En una terminal ejecuta `ollama pull qwen2.5:1.5b`. Cuando finalice, verifica `ollama list`. Edita `.env` y cambia `CATIA_PROVIDER=ollama`. Mantén `CATIA_RETRIEVER=lexical` para una primera ejecución simple. Reinicia CAT-IA. La etiqueta de configuración debe indicar LLM; la prueba real es enviar un ticket y comprobar `status=draft` con uso de tokens y citas, no solo ver `/api/health`.

Si Ollama no está escuchando, inicia su aplicación o utiliza `ollama serve` en otra terminal cuando corresponda. La app usa `http://127.0.0.1:11434`. El modelo debe coincidir exactamente con `OLLAMA_MODEL`. La descarga inicial requiere internet; luego el modelo puede ejecutarse localmente. No se necesita una API key para esa ruta.

Recorrido de uso

1. Escribe qué pasó y el sistema afectado. Evita datos personales reales.
2. Selecciona si afecta a una persona, equipo u organización. Marca interrupción y seguridad únicamente cuando corresponda al escenario.
3. Deja cuatro fragmentos y fuentes internas y externas para comenzar.
4. Pulsa Analizar con evidencia y espera. En modelo local puede tardar según el equipo.
5. Lee la categoría y el resumen, y verifica la prioridad contra el impacto declarado.
6. Lee el borrador y abre cada fuente. Compara que las recomendaciones estén realmente respaldadas; un extracto exacto por sí solo no basta.
7. Consulta el recorrido para distinguir recuperación, generación, validación y abstención.
8. Descarga JSON si necesitas guardar evidencia. La app no envía la respuesta a ningún cliente.

Ejemplos para practicar

Caso 1: botón Acceso web. Debe recuperar una guía externa de cookies y procedimiento interno relacionado. Caso 2: Cobro duplicado, individual y sin interrupción. Se espera Facturación y Baja, sin devolución aprobada. Caso 3: mismo caso con una caída de toda la organización declarada; discutir por qué el operador debe comprobar que esos metadatos sean coherentes. Caso 4: Sin evidencia, astronomía; se espera abstención en recuperación lexical.

Estados

draft: salida con contrato y citas validadas, aún pendiente de revisión humana. insufficient_context: no hay evidencia sobre el umbral; solicitar antecedentes o derivar. invalid_output: el modelo no devolvió estructura/citas válidas. provider_error: no se pudo completar generación, por ejemplo por conectividad. HTTP 503: falló recuperación, posible problema de embeddings. Cada uno requiere un diagnóstico diferente.

Cambiar conocimiento y parámetros

Edita una fuente Markdown, actualiza version en data/sources.json, verifica su contenido y ejecuta `.venv/Scripts/python.exe -m scripts.source_hashes`. Reinicia el servidor para reconstruir el índice. Para una fuente nueva agrega entrada con id único, file, title, type, origin, version y active. Si es simulada, indícalo. Para una fuente externa guarda una síntesis propia breve y referencia su URL y fecha; no marques como consultada si no la revisaste.

Para cambiar fragmentación, umbral o versión del prompt edita config/rag.json y reinicia. Cambiar top-k en la pantalla afecta solo esa consulta. Cambiar source_filter no elimina la política interna de prioridad. Para modificar SLA, actualizar catia/agent.py, data/internal/sla.md y pruebas de prioridad conjuntamente.

Problemas frecuentes

- Python no se reconoce: instalar Python y reabrir la terminal.
- Puerto 8000 ocupado: usar `.venv/Scripts/python.exe -m uvicorn catia.api:app --host 127.0.0.1 --port 8001` y abrir ese puerto.

- Hash distinto: hubo un cambio de fuente; revisar antes de regenerar hashes.
- LLM configurado pero falla: comprobar Ollama, modelo descargado, memoria y endpoint. CAT-IA no oculta ese fallo con demo.
- /docs no carga sin internet: consultar /openapi.json; Swagger usa recursos CDN.
- Respuesta poco relevante: revisar fuentes recuperadas antes de editar el prompt. Aumentar k indiscriminadamente puede agregar ruido.
- Embeddings tardan al arrancar: se indexa todo el corpus. Para volver a baseline, configurar CATIA_RETRIEVER=lexical y reiniciar.

Preparación del día de presentación

El día anterior, ejecutar pruebas y una inferencia real en el mismo computador. Dejar modelos descargados, PDF a mano y ejemplos preparados. Llevar resultados anteriores identificados como evidencia guardada, sin fingir que son inferencias en vivo. Si falla el modelo, explicar la falla y mostrar demo solo como recorrido de interfaz, reconociendo que no cumple la parte de inferencia en vivo.